Cloud SDK for Scotty: Cloud SDK Libraries update requirements

Status: Draft > Review > Current > Needs update > Obsolete
Link: go/cloudsdk-scotty-requirements
Authors: Viacheslav Rostovtsev
Last updated: Jul 13, 2026
See also: Cloud SDK for Scotty: Universal Resumable Upload Protocol
Cloud SDK for Scotty: the API team's and the end-user's CUJs

Username	Role	Status	Last change
coryan	Approver	🟡 Pending	Jul 13, 2026
alexghi	Reviewer	🟡 Pending	Jul 13, 2026
atarafamas	Reviewer	🟡 Pending	Jul 13, 2026
betterbrent	Reviewer	🟡 Pending	Jul 13, 2026
blakeli	Reviewer	🟡 Pending	Jul 13, 2026
gabepearhill	Reviewer	🟡 Pending	Jul 13, 2026
mzp	Reviewer	🟡 Pending	Jul 13, 2026
partheniou	Reviewer	🟡 Pending	Jul 13, 2026
sdhart	Reviewer	🟡 Pending	Jul 13, 2026
suzmue	Reviewer	🟡 Pending	Jul 13, 2026

Approval Instructions: Please approve or LGTM through the G3 Assist sidebar.
For more information, see go/g3a-approvals-reviewing

Summary

Scotty (go/scotty, go/scotty-http-protocols) is a shared Google service for handling large HTTP/HTTPS uploads and downloads. Scotty acts as a frontend that offloads the complexity of file transfers from individual product backends. Google API teams using Scotty implement a "customer agent" backend that Scotty notifies about transfer progress and completion, allowing the team's application to manage the file and control access.

Scotty implements multiple custom protocols for data upload, some with legacy versions. For data download Scotty acts as a "reverse proxy" to the backend data sources, supporting standard HTTP protocol with Range requests.

This document describes adding support for Scotty in Cloud SDK Client Libraries. It currently describes **only** adding support for uploads, with design for downloads support to be added later. Only one protocol is in scope for this work – Unified Resumable Upload protocol. Full discussion of scope is in the "Scope" section of this document.

Changes in Client Libraries require supporting changes in other Cloud SDK code. This document covers centralized changes in API Publisher and Gapic Config, as well as changes in Common "GAX" Libraries, and Gapic generators, in addition to Client Libraries changes.

A note on terminology

I will be using the term "Scotty" throughout this document in the following contexts:

- Scotty annotation – `google.api.http` annotation, with the `resumable_upload` option.
- Scotty protocol (or just "upload protocol") – Unified Resumable Upload protocol.
- Scotty RPC – a proto RPC with the Scotty annotation
- Scotty method – a method (or a set of methods) corresponding to a Scotty RPC in the Client Libraries' code.
- Scotty-containing service – a proto `service` containing a Scotty RPC.
- Scotty-containing API – a Google API (Cloud or non-Cloud), containing a Scotty service.

Cloud SDK terminology:

- "GAX", "GAX Library" – Primary dependency for all generated libraries. Can be named differently (e.g. "api-core" in Python).
- Veneers (or handwritten) – "Vertical SDK" libraries that depend on GAX and/or Gapic-generated clients. "Vertical SDK" libraries that are entirely dependent on Apiaries are out of scope for this document entirely, and the term "Veneers" does not include them.
    - Veneers are out of the scope for the *Client Libraries requirements* but this document still talks about them descriptively.

Scope

Upload scoping

Aside from several generic points that mention the downloads, the requirements in this document are scoped to Resumable Uploads only. This is because there is no end-to-end protocol for the Downloads, and the behavior of the Download is unclear at the moment.

However: I strongly believe that we're not cutting any meaningful design space for Uploads by implementing Downloads with the requirements below. I think that "Same method-with-helper" model will work for both, and the only potential issue will be that the parameter sets for Uploads and Downloads are a little different, which is an acceptable tradeoff for not blocking on the full Upload design.

Scotty protocols in scope

Unified Resumable Upload protocol: current as of CL 910837347 (2026-05-05). (https://g3doc.corp.google.com/cloud/storage/g3doc/team/sub_teams/servingfabric/sub_teams/serving_frontend/sub_teams/protocols/resources/documentation/scotty/http_protocols.md?cl=910837347#unified-upload-protocols).

See Cloud SDK for Scotty: Universal Resumable Upload Protocol for the discussion of the upload protocol from the Client Libraries perspective.

There will be a separate document for Download support. This document focuses on the Upload only.

Scope for Cloud SDK Libraries and Generators

In Scope for the Client Libraries requirements (CL-Rx): Every generated library in Cloud SDK languages. Veneer libraries are NOT in scope for this project. If any Veneer libraries decide to update to use the GAX implementations of the Scotty protocol they SHOULD use the Client Libraries requirements (labeled CL-Rx) as guidelines.

In Scope for the GAX Libraries requirements (GAX-Rx): Every common library in Cloud SDK languages.

In Scope for the Generators requirements (GEN-Rx): Every gapic generator in Cloud SDK languages.

Apiary libraries are not in scope

Apiary has existing Scotty support in some languages; furthermore, Scotty is feature work and is exactly the kind of project that should not be extended to Apiary.

Project milestones

Milestone 1: deliver Scotty Upload functionality for Google Ads "googleads"

Google Ads "googleads" libraries also known as "Simply libraries" are the largest Google Ads libraries we generate. They have recently added one method in one service (`rpc CreateYouTubeVideoUpload` in `service YouTubeVideoUploadService`) which is a Resumable Upload Scotty method.

The requirements in scope for Google Ads "googleads" are:

- Implement Scotty Resumable Upload, no Download required.
- Use allowlist for Scotty detection, no annotation parsing or gapic-config parameters.

The deadline for Milestone 1 is EoQ3, with the exception of select languages.

Milestone 2: deliver Scotty Upload functionality for all libraries generated by Cloud SDK (Google Cloud and other orgs)

This is a full implementation, which includes all requirements in this document.

The deadline for Milestone 2 is EoQ3, same as Milestone 1.

Milestone 3: deliver Scotty Download functionality for all libraries generated by Cloud SDK

Milestone 3 will be covered by a separate document.

Overview

Scotty strategy for Cloud SDK Client Libraries:

- One set of protocols: latest version of the Unified Resumable Upload protocol.
- Scotty annotation (a property in `google.api.http`) published as a marker for the Scotty Rpcs.

- Generation opt-in for APIs with a parameter in the `ClientLibrarySettings` config.

Supporting work

There are two work items that are complementary to Libraries requirements. They are described here, but they are not requirements for Cloud SDK Language teams.

Publish Scotty annotation

Scotty annotation is not a separate annotation, but two messages in google/api/media.proto: `MediaUpload` and `MediaDownload`. Both of these messages contain an `enabled` boolean delineating whether the upload/download functionality is enabled. I propose to publish these messages, with only this field in those messages as a content, so the client-side `CreateYouTubeVideoUpload` method will look like this:

Protobuf

```
rpc CreateYouTubeVideoUpload(CreateYouTubeVideoUploadRequest) returns
(CreateYouTubeVideoUploadResponse) {
option (google.api.http) = {
      post: "/v1/media/create_youtube_video:upload"
      body: *
      media_upload: {
        enabled: true
      }
    };
}
```

NB: the `MediaUpload` message has a `resumable_upload` field but it only controls whether/how the discovery docs are updated. Therefore we should not use it as a capabilities marker and will not publish it.

Add an option to ClientLibrarySettings for configuring Scotty generation

We need to allow the Generators to read the prefix for the Scotty initial URI.

I propose to add a Scotty submessage in the `ClientLibrarySettings` message that will (1) serve as an opt-in for Scotty generation, and (2) serve as a source of truth of the prefix for the Generators:

Protobuf

```
message ScottySettings {
```

```
  // Whether the Scotty generation is enabled for the API.
  // If this is not enabled or if the Scotty settings are not present
  // in the API configuration, the code for the Scotty methods should not be
generated.
  bool upload_enabled = 0;
  // Prefix that is appended to RPC URIs for the initial Scotty request
  string upload_prefix = 1;
}
```

There are other settings that might be implemented later for Scotty integration, e.g. whether to calculate and send the checksum for the upload by default.

Won't these settings be per-RPC?

All APIs I talked about have at most one upload/download method. If they add more, I imagine the prefix will be the same (as will be other Scotty server-side configuration options such as checksuming). If we see an API with multiple differently configured methods we'll add the per-RPC submessage to `ScottySettings` but I don't want to add a complex configuration while it's not needed.

Alternative: add the per-RPC setting to Scotty annotation

OnePlatform owns the Scotty annotation and Scotty team mentioned that changing it is challenging. Therefore I think having the Scotty settings in the `ClientLibrarySettings` is the best available option.

The implementation details

Alternative considered: supporting Multipart protocol

A common question is whether we should support the Unified Multipart protocol for smaller files. The Multipart protocol saves allows putting the data and the request message in the same request, saving one roundtrip between the client and the server. I think that supporting it is a potential future improvement for Client Libraries, but it is very much outside the scope of this Scotty project. Additionally I don't think that we are cutting off any meaningful design space for future Multipart functionality with the current Scotty design.

Constructing the Scotty initial URL using the configurable prefix

Scotty methods must send the initial request to the special URL. The special URL is created by prepending a configurable prefix to the "verb" URL in `google.api.http` annotation, e.g. the

prefix for the GoogleAds simply libraries is `resumable/upload`, so the URL is changed as following:

`https://{host}/v24/customers/{id}/youTubeVideoUploads:create` -> `https://{host}/resumable/upload/v24/customers/{id}/youTubeVideoUploads:create`.

This prefix is configurable in the server-side Scotty configuration, although I expect for most services it will be either `/resumable/upload` or `/upload`. However it is *not* surfaced in the Scotty annotation for the RPC (and changing the annotation is challenging). Instead the Generators **must** get it from a submessage in the `ClientLibrarySettings` generation option and insert it in the generated libraries code.

Setting Scotty upload configuration; Scotty upload helper

A Scotty upload helper is a separate `ResumableUploadHelper` class that the end-user can utilize to set the configuration of the upload. Creating the class and exposing instances of it, and its public surface to the end-user is mandatory (**must**). This is to ensure that the libraries in all languages present similar UX to the end-user.

The problem surface we are working with is:

1. The end-user needs to be able to provide the familiar arguments to the surface. These include request message, optional retry policy, optional headers.
2. The end-user needs to be able to provide some amount of Scotty-specific arguments. At the minimum: the stream, the chunk size.
3. We need to keep this extensible against potential future scope expansion. The encoding, the checksum type, the chunk deadline.
4. We need to offer an additional surface for resuming the upload after an error or pause.

There are multiple solutions that potentially fit, e.g. everything can be an optional parameter on the Scotty method, or there can be multiple overloads of Scotty methods. But the most practical solution that is available in all languages and can be made to fit the natural style is to expose an `ResumableUploadHelper` class.

An instance of the `ResumableUploadHelper` class then can be returned from a Scotty method, and allow the parameter set-up. After the setup the upload can be started with a separate call to a `begin-upload` method on this object. Since the only mandatory parameter for Scotty is the end-user-provided data object, it is fairly natural for this `begin-upload` method to take the data object as a mandatory parameter.

The code might look like this:

```
Kotlin
var client = new YouTubeVideoUploadServiceClient();
```

```
// regular-looking method call
var upload_helper = client.CreateYouTubeVideoUpload(request, headers);

// additional configuration
upload_helper.set_chunk_size(chunk_size);

// upload starts
var response = upload_helper.begin_upload(stream);
```

The `ResumableUploadHelper` class **must** be made public, however its constructor **should not** be.

The download helper will be a separate class, therefore the name of the upload helper **must** include the words "resumable" and "upload". E.g. `ResumableUploadHelper`, and not just `Helper`. This is language-specific and left to the language implementations.

Timing of the initial request

The timing of the initial request to the server-side API can be during the call to the Scotty method (`CreateYouTubeVideoUpload` in the example above) or when the `begin-upload` method is called. This is language specific.

The advantage of delaying the call is:

- The end-user can call the Scotty method on the Client Libraries' `Client` class to get an instance of the upload helper class, and then call the `resume-upload` method on that instance. Since the initial request is not sent when the Scotty method is called, this works for resuming an upload.
  If, on the other hand, the initial request is sent when the Scotty method is called, the end-user must have another way to obtain the upload helper, e.g. by creating an instance of the class directly. This increases the public surface that we need to give to the end-user.
- The execution can transition to the protocol implementation once, with a full set of parameters, instead of separating the `start` command. The alternative way of saying the same thing is: the `start` command can remain part of the protocol implementation in GAX instead of being singled out as a part of the Client Libraries generated code.
- The progress tracking will be set up when the initial request is sent, and the end-user can be notified of the progress immediately.
- If we ever encounter a capability that needs to be reflected in the initial call we won't have to change the code. Right now the only capabilities like that are in the headers. An

example of a potential future client-side capability like that is changing the protocols (at user's request or because the upload size is small).

The advantage of making the call right away is:

- The Scotty upload URL can be made returnable from the helper's surface immediately.
- Symmetry with the pagination and the LRO helpers where the initial call happens immediately

Alternative considered: add the upload configuration to the method parameters

This is suggested by Amanda Tarafa Mas.
The code might look like this:

```
Kotlin
var client = new YouTubeVideoUploadServiceClient();

var upload_options = new UploadOptions();
upload_options.set_chunk_size(chunk_size);

var upload = client.CreateYouTubeVideoUpload(request, headers, upload_options,
stream);

// upload starts
var response = upload.begin_upload();
// or
var response = upload.resume();
```

I think this is worse because when the end-user passes the stream they might expect the upload to start executing. However it has the advantage of not having the helper be mutable.

Resuming the upload

The end-user **must** be able to resume an upload after it was interrupted. The Scotty upload helper **must** be utilized for that purpose to present the unified experience to the end-user.

The mechanism for this is left to the language implementations. The recommended way is to

1. Not make the initial `start` command call when the end-user calls the Scotty method and the helper is created.
2. Expose a separate `resume-upload` method, that would take as the required parameters the end-user-provided data object and the upload URL.

In this way, since the additional method is exposed on the `ResumableUploadHelper` class public surface, and **not** on the Client Libraries' `Client` class public surface, we don't have the potential of the naming conflicts.

Note that we **do not need** the upload offset from the end-user. The server is the source of truth on how much data was uploaded. The protocol implementation **must** start the upload resume by querying the server for the offset, seek-ing the stream to that offset, and buffering the next chunk.

In most cases the end-user should provide the data object (Stream) rewound to the offset 0, and the documentation **should** reflect that. If the end-user provides a non-seekable stream rewound to the offset that is after the server's source of truth for bytes committed, the protocol implementation will raise an error.

Additionally, if checksuming is enabled, passing a non-rewound stream **must** raise an error.

Including the Scotty surface in gRPC and REST surfaces; using the Mixin mechanism

The requirements for including the Scotty surface in existing in gRPC and REST surfaces are written in general language, and marked as **must**. Using the mixin mechanism for that purpose is recommended, but optional. Mixin-specific suggestions are marked as **should**. This is because the setup for "surfaces" is different in all languages.

To elaborate, the Client Libraries currently expose two public API surfaces, gRPC and REST. Alternatively, they expose one API surface, and the "gRPC or REST" is an option that the end-user passes when creating a client for that surface. That option can be called "transport" and the "transport" setup typically has gRPC and REST sub-surfaces, or "transports".

REST-Resumable surface is a subset of the API surface that includes, without loss of generality, a small subset of RPCs within a Scotty service. It is convenient to generate it "on the same level" as gRPC and REST surfaces (however those are generated). This is because:

1. There is a specific set-up involved when calling Scotty RPCs so there is a non-trivial complexity to generate.
2. The requirement for Scotty methods to be functional on the gRPC surface means that generating Scotty as part of the REST surface will make the setup asymmetrical: REST surface will get the Scotty-specific setup complexity and gRPC surface will get the "must pipe the call to the REST surface somehow" complexity.

When generating the REST-Resumable surface separately,

1. The Scotty-specific setup is contained to the Scotty templates, and

2. The "must pipe the call to the REST-Resumable surface" complexity is the same in both REST and gRPC.

And we already have a "must pipe the call to a different generated surface" abstraction in the mixin mechanisms used for LRO (and locations/iam, although the variance is larger for those).

For these reasons generating the REST-Resumable surface separately and including it as a mixin is recommended (**should**).

The REST-Resumable surface **should not** be a public surface. Instead we want existing public gRPC and REST surfaces to contain the public-facing methods for Scotty RPCs. The best way to satisfy this requirement will be language-specific.

The REST-Resumable surface **may** be shared for uploads and downloads.

The naming of the surface is left for the language team. In the event the REST-Resumable surface has to be made public, the name **must not** include the word "Scotty".

Creating the REST-Resumable surface: the gRPC Channel

One issue exists for creating the REST-Resumable surface: if the gRPC surface is initialized with the gRPC channel in place of credentials:

```
Kotlin
// the end-user code
var client = new YouTubeVideoUploadServiceClient(transport: grpc, credentials:
grpcChannel);

// inside the constructor for the YouTubeVideoUploadServiceClient:
var restResumableMixin = new  YouTubeVideoUploadServiceRestResumable(credentials:
???)
// grpcChannel does not work since there is no way to extract credentials from
that
```

The Libraries **must NOT** raise an error here, since doing this means that adding Scotty RPCs and enabling Scotty generation has potential to break customers who are not using Scotty.

Instead the Libraries **must** initialize the gRPC client surface normally, and raise an error only when (if) the end-user calls the Scotty method.

If the Libraries show warnings to the end-user, the Libraries **should** show a warning message informing the end-user that the resumable upload/download methods will not work when the client surface is initialized with a pre-made gRPC channel.

Mixin example code for the LRO (Ruby)

This example code, for the LRO in Ruby, consists of 4 parts:

1. The LRO surface, generated separately

File: `service_name/operations.rb`

```
None
module ServiceName
      # Service that implements Longrunning Operations API.
      class Operations
       <...>
      end
end
```

2. The `require` statement making sure the LRO surface is available to the Client Library's Client class

File: `service_name.rb` (declares module for the ServiceName service)

```
None
require "service_name/operations"
require "service_name/client"

module ServiceName
<...>
end
```

3. The LRO surface client creation that includes passing relevant parameters

File: `service_name/client.rb`

```
None
module ServiceName
  class Client
    def initialize
      <...>
      @operations_client = Operations.new do |config|
        config.credentials = credentials
        config.universe_domain = @config.universe_domain
        <...>
```

```
      end
      <...>
    end
    <...>
  end
end
```

4. The call to LRO surface's method within the Client Library's Client generated method, exposing the `::Gapic::Operations` LRO Helper to the end-user

File: `service_name/client.rb`

```
None
module ServiceName
  class Client
    def lro_method request, options = nil
      <...>
      @service_stub.call_rpc(:lro_method,
                             request,
                             options: options) do |response, operation|
         response = ::Gapic::Operation.new(response,
                                           @operations_client,
                                           options: options)
      <...>
      end
    end
    <...>
  end
end
```

The Ruby constructions are idiosyncratic, but essentially a response from the initial RPC call is wrapped in the `::Gapic::Operations` LRO Helper object. Subsequently that helper object will be exposed to the end-user via code hidden in `<...>` blocks.

Tracking the upload progress; providing a callback parameter for the end-user

The requirements for tracking upload progress are written in general language, and marked as **must**. Using a callback mechanism for that purpose is recommended, but optional. The callback-specific suggestions are marked as **should**. This is because languages have different preferences for organizing the end-user's interactions with an independent continuous process, such as chunked upload (or download).

The purpose of this mechanism is to allow the end-user to receive progress notifications (data uploaded, state of the upload).

The upload progress data structure that the end-user receives **must** contain at least the following:

- Amount of data that the server has confirmed as uploaded (via the well-formed HTTP 200 replies to the `upload`, `upload,finalize`, or `finalize` commands).

Sending state of the upload information with the upload progress

It is recommended that the upload progress data structure **should** contain:

- State of the upload, typically as an enum, with the following values: "started", "uploading", "recovering", "recovered", "finalized".

The states must be sent as follows:

- "started" state must be sent after a successful reply to the `start` command.
- "uploading" state must be sent after a successful reply to the `upload` command.
- "recovering" state after a "recoverable" error reply to `upload`, `upload,finalize`, or `finalize` commands.
- "offset received" state after a successful reply to the `query` command.
- "finalized" state after the successful reply to the `upload,finalize`, or `finalize` commands.

In this way all the "successful path" state transitions in the successful upload path are covered in the progress tracking. If during the `query` the progress will be reset or move forward to a point in the stream that is different from the chunk size, the end-user will see the "explanation" in the progress notification.

NB: implementations that elect to not delay the initial call until the end-user has completed the setup will not be able to notify the end-user of the "starting" state immediately after a successful reply to the `start` command. They **may** elect to send the "starting" state upload progress notification before the first `upload` command instead.

Making the upload URL available with the progress notification

It is recommended that the upload progress data structure **should** contain:

- The Scotty upload URL that the server returns with the successful reply to the `start` command.

This is simply for the convenience of the end-user, and so that the upload progress data structure can contain all the data that is needed to continue the upload if it is interrupted.

Future-proofing: pause and cancel functionality

If the end-user encounters an error that does not lead to the server rejecting the upload (e.g. a networking error, or error when reading their data), they can resume the upload at a later time. As a stretch goal, we **may** allow the end-user to pause the upload without raising an error.

Pausing is a public API interaction (e.g. a method), and the precise mechanism is left to the language implementations. While pausing is a **may** (pending requests from API teams), ensuring that there is a place where this pause capability can be implemented is a **must**.

A suggested method to implement the pause is on the progress callback, if one is used for the upload progress tracking. E.g. the callback can have a return value where the end-user can set a flag requesting that the upload is paused.

Cancelling the upload is the same as Pausing: the capability itself is a stretch goal (**may**) while ensuring that there is a place where it can be implemented is a **must**.

The requirements format

The code of the Client Libraries is tightly coupled with GAX requirements. This means that a lot of requirements describe functionality that sits somewhere between the Client Library code and the Scotty Helper code (which is in GAX). To avoid repeating myself twice for most requirements I have written everything I could through the lens of the Client Libraries (e.g. "The Client Libraries must").

There are requirements where that language did not fit:

For GAX:

- requirements specific to the internals of the protocol implementation, e.g. buffering
- requirements specific to the visibility of the GAX components

For Generators:

- requirements specific to reading Scotty annotation or configuration.

Baseline requirements (CL)

CL-R0 Client Libraries **must NOT** use the word "Scotty" in method names, helper classes names, configuration parameters, documentation to the aforementioned, README files, public-faced commit messages, changelogs, etc.

"Scotty" is an internal designation. The words "Upload" or "Resumable Upload" **must** be used instead, e.g. if a Scotty upload helper is mentioned in documentation, it **must** be described as a "resumable upload helper".

CL-R1 The Client Libraries for a Scotty API **must** contain a Scotty method or methods corresponding to Scotty RPCs in the services of this API, for both gRPC and REST surfaces

This means that the gRPC clients **must** bring in REST dependencies for a Scotty API. In some languages this can be done dynamically, so that the dependencies are loaded on-demand at runtime.

This requirement is especially important for "googleads" libraries since they only generate and use gRPC code (except for PHP), but even in the libraries for which both gRPC and REST surfaces exist, the gRPC surface **must** contain methods for Scotty Rpcs.

The recommended way to do this is via the mixin mechanism. Generating the code for Scotty API **should** generate a separate API surface containing only Scotty methods, and that surface **should** be referenced in both gRPC and REST clients in the same way. The setup **should** be similar to e.g. LRO or Location mixin.

CL-R2 The Client Libraries **must** allow the same methods of initialization for the services with Scotty RPCs, as for the services without Scotty RPCs.

This is important for backward compatibility. For an API team, adding a Scotty RPC to an existing service must NOT be a breaking change. This however means one significant exception where it will be impossible to call the Scotty methods from a gRPC client surface service.

CL-R2.1 When a gRPC client surface for a Scotty API is initialized with a pre-constructed gRPC Channel, the Client Libraries **must NOT** raise an error during initialization.

Once again, this for backward compatibility. Even though it will be impossible to call a Scotty method with this form of initialization.

CL-R2.2 When a gRPC client surface for a Scotty API is initialized with a pre-constructed gRPC Channel, the Client Libraries **must** raise a detailed (actionable) error if a Scotty method is called.

The error **should** explain that a client surface was initialized with a gRPC channel, that credentials cannot be extracted from a gRPC channel, and that the Resumable uploads/downloads happen over REST and cannot use gRPC channels as credentials. It **should** also recommend that the end-user provides the credentials in place of a gRPC channel if they want to use Resumable upload/download functionality.

CL-R3 Scotty upload methods in Client Libraries **must** have the signatures typical for a generated unary method, except that they **must** return a Scotty upload helper

The name of the method(s) **should** be the same as it would be for a unary method(s), and it **must** take the same set of parameters. The return type **must** be a Scotty upload helper, or an intermediate mechanism to get said helper, e.g. a `Builder` or a `Factory`.

CL-R4 The Client Libraries **must** use the Scotty initial URL prefix to form a URL for the Scotty `start` command

The upload prefix is a string, e.g. `resumable/upload`. It is used to form the Scoty initial URL, to which the protocol implementation will send the `start` command. The details on how to form the URL are in [TK] section.

CL-R5 The Client Libraries **must** return an instance of the protobuf-defined response message to the end-user

The last message from the server will have the response message in the JSON format. The Client Libraries or the Scotty Helper methods **must** decode it and return it as the protobuf object, same as they do for the methods on the REST surface. The separation of the responsibilities is left to the language implementations.

Requirements that describe Scotty upload configuration (CL)

CL-R6 Scotty upload methods in in Gapic-generated Client Libraries **must** allow the common set of parameters to be configurable in the same way that they would be configurable for a generated unary method

This is separate from the method signature requirements because some languages allow configuration via e.g. a `Configuration` setter on the Client Libraries' `Client` object, and NOT via the parameters in the method signature.

These common configuration parameters are in contrast to the Scotty-specific parameters, which are discussed below.

Common configuration parameters include at least the following:

- The request message
- The optional headers
- The retry policy
- The deadline or timeout, which might be a part of the retry policy.

These parameters **must** be passed to the protocol implementation in GAX, via the Scotty upload helper.

They are to be used as follows:

- The request message – sent as a body with the HTTP request containing the initial `start` command
- The optional headers – sent with the HTTP request containing initial `start` command
- The retry policy – used as an transient retry configuration for `start` command
- The deadline or timeout – used as a local deadline for the `start` command

CL-R6.1 The generated documentation for the Scotty upload methods and configuration **must** be updated to reflect the scope of the common configuration parameters

The generated documentation for the parameter configuration, whether it's attached to the Scotty methods or somewhere else **must** be updated to explicitly state the scope of the configuration, e.g.:

- That the optional headers will only be sent with the initial request.
- That the retry policy is only applicable to the initial request.
- That the deadline or timeout, will be used as the deadline for the initial request.

CL-R7 The Client Libraries **must** expose a mechanism for the end-user to provide an additional Scotty-specific configuration for the Scotty uploads

The easiest way to "expose a mechanism" is to return an instance of the Scotty upload helper with e.g. the public setter methods. But e.g. returning a Builder with setter methods is also acceptable.

Please note that the end-user still **must** be able to get a Scotty upload helper instance (CL-R2), either from the initial call, or at the end of the additional configuration process.

The Scotty-specific configuration parameters include:

- The end-user-provided data object (Stream)
- The global deadline/timeout
- The chunk size for the Scotty upload
- The Scotty upload URL (when resuming)

They are to be used as follows:

- The end-user-provided data object (Stream) – used as the source of the data to upload.
- The global deadline/timeout – used as the global deadline, or to calculate the global deadline for the upload.
- The chunk size for the Scotty upload – used as the buffer size, and the size of the data chunk in the Scotty upload request, unless negotiated down by the server with the granularity requirements.
- The upload URL – used when resuming an upload, as a URL to send the protocol commands to.

The end-user-provided data object **should** be a required parameter to the Scotty upload helper's `begin-upload` and `resume-upload` methods.

The upload URL should be a required parameter to the Scotty upload helper's `resume-upload` method.

CL-R7.1 If the Client Libraries provide a name (e.g. for a class or an interface) for the Scotty-specific configuration, that name **must** be upload-specific

E.g. the name **should** be `ResumableUploadOptions`, not `TransferOptions`. The Client Libraries **must NOT** attempt to merge Upload and Download options in a generic class, as they are expected to be substantially different.

CL-R8 The Client Libraries **must** reserve a namespace for the end-user to supply additional future configuration parameters for Scotty uploads

The namespace **must** be in the same mechanism as used for the Scotty-specific configuration parameters, such as chunk size. This should be a no-op for all implementations, since we can always add another appropriately-named setter to the Scotty upload helper, or another `WithX` method to the upload helper builder.

The required additional configuration parameters include:

- Size of the upload (bytes) – can be used to better calculate the global deadline
- Per-chunk timeout (seconds) – used as a local deadline for the phases sending `upload` and `upload,finalize` commands.
  The current behavior is to use a sensible default for all local deadlines, except possibly for the initial request containing the `start` command.
- Encoding (string) – the data encoding type, used to indicate to the server that the end-user uploads compressed data.
  The current behavior is to not specify any encoding.
- Checksum type (enum) – the type of the checksum, used to select the checksuming algorithm on the client, and to create the checksum header.
  The current behavior is to not calculate the checksums, and not send the checksum header.

As mentioned, this should be a no-op for all implementations.

Requirements that describe reasonable defaults for configuration parameters (CL)

CL-R9 The Client Library **must** provide reasonable defaults for (most of the) Scotty configuration parameters.

The configuration parameters that we cannot provide the defaults for are:

- Request message
- The end-user-provided data object (Stream)

The configuration parameters that the reasonable defaults **must** be provided for are:

- The optional headers – default: **none**
- The retry policy for the HTTP request containing initial `start` command – default: **none**, to be decided by the protocol implementation
- The local deadline for the HTTP request containing initial `start` command – default: **none**, to be decided by the protocol implementation
- The global deadline. A reasonable default **must** be provided, by one of the following methods:
    - If the API team has configured it with the `timeout` parameter in the gRPC config, that value should be used (inserted by the generator)
    - Otherwise, the same reasonable default as for any other method in the library **must** be used.
- The chunk size. A reasonable default **must** be provided, e.g. 10 megabytes.

CL-R9.1 The Client Library **may** estimate better reasonable defaults.

The client library **may** contain a reasonable default for upload speed to Google Cloud, e.g. 10 megabit/s.
Then the client library **may** estimate a local timeout for chunk upload by using that speed together with the chunk size by dividing the chunk size by the speed, and then accounting for the retries.

Likewise, if the client library allows the end-user to specify a total upload size, it can use the speed estimate to calculate a better global deadline default.

The details of the calculation are left to the language team, since this is an optional requirement.

Behavioral requirements (CL)

CL-R10 The Client Libraries **must** accept a language-specific Stream type for the end-user-provided data object; or a similar generic abstraction, e.g. a commonly used IStream interface.

The Client Libraries **may** offer additional overloads of the method, such as accepting a `File` object. Alternatively, it **may** provide a wrapper over a `Stream` abstraction for the end-user's convenience.

The Client Libraries **should NOT** accept a gRPC-specific streaming interface or abstraction, since Scotty and gRPC streaming are completely distinct.

It is possible that the common `Stream` abstraction is unseekable (or whether a given instance is seekable can only be discovered at runtime). If required to seek an unseekable `Stream` abstraction (for recovery)(, the library **must** raise an error.

As a reminder, the end-user-provided data object is a Scotty-specific configuration parameter, and it is recommended that it is implemented as a required argument for the `begin-upload` method on a Scotty upload helper.

CL-R11 The Client Libraries **must** allow the end-user to track the upload progress

The end-user **must** be able to configure the protocol implementation such as, after the server replies with confirmation of the data chunk upload they get notified of at least the amount the bytes uploaded.

The recommended way to do this is via a callback function. The Client Libraries **should** allow the end-user to pass a callback function that will be called by the protocol implementation at least after every confirmation of the data chunk upload.

The Client Libraries also **should** notify the end-user of the protocol state, e.g. recovering, starting, and finalizing.

See the "Tracking the upload progress" section for details.

CL-R12 The Client Libraries must allow the end-user to get the upload URL

For a given Scotty upload, the server generates a unique upload URL and returns it in response to the `start` command. All commands after the `start` command are sent to that upload URL. The Client Libraries **must** allow the end-user to get this URL, as it is needed for resuming the upload in case of an error.

The Client Libraries **should** make the upload URL available as a getter on a Scotty upload helper.

The Client Libraries **should** include the upload URL as part of the errors it returns to the end-user when Scotty-related errors occur.

The Client Libraries **should** include the upload URL as part of the upload progress tracking information.

CL-R13 The Client Libraries **must** allow the end-user to resume the unfinished upload, and to provide parameters required to do so

The Client Libraries **must** use the Scotty upload helper when the end-user resumes the download, so that the experience is identical, except for the initialization. For the initialization, the Client Libraries **must** use an additional end-user-provided parameter: the upload URL. This URL must be provided to the protocol implementation, and it must use it in place of the upload URL it would usually get from as a response to the `start` command.

The mechanics of resuming the upload are discussed in the "Allowing the end-user to resume the upload" section.

CL-R13.1 The documentation for resuming the upload in the Client Libraries **should** explain the potential of error if the stream is unseekable and not rewound to 0

It is easy for the end-user to have misconceptions about resuming the upload. It is OK if the end-user provides a non-rewound stream, as long as it is seekable. It is similarly OK if the end-user provides a non-seekable stream, as long as it is rewound to 0.

However if the end-user provides a stream that is both unseekable and non-rewound, they are likely to encounter an error, e.g. if they are recovering after an error and the server has not saved the last data chunk.

The Client Library documentation **must** explain the potential for error and recommend that the stream is rewound to 0 when provided for the purpose of resuming the upload. The details are left for specific implementation.

CL-R14 The Client Libraries **must** reserve the namespace for the end-user to pause and cancel the upload

The place for this functionality is language-specific, since it requires interacting with the ongoing upload.

The recommended place for it is the user-provided callback. The Client Libraries **should** include a return type for the end-user provided callback, so that the end-user can pause or cancel the upload by setting e.g. a property in that return type. (NB: This is similar to the recommendation that proto RPCs return a type, instead of `google.protobuf.empty`).

CL-R15 Observability: The Client Libraries **must** provide at least the same level of observability for Scotty methods as they provide for all other methods

This is a broad requirement that is essentially: don't throw observability out of the window just because it's a Scotty method. This is very language-specific, and is left entirely to the language implementations.

This requirement is out of the scope for Milestone 1 ("googleads").

CL-R16 Actionable Errors: The Client Libraries **must** present actionable errors for Scotty-specific error modes

This means that the Client Libraries **must** return an error of an appropriate Cloud-specific or Ads-specific error type, which wraps an underlying exception if any, and has a descriptive error message.

The Scotty-specific error modes are:

- Attempting to initiate a Scotty upload after attempting to initialize the Rest-Resumable surface with a "gRPC Channel" credentials
- Attempting to seek an unseekable stream
- Broad language-specific class of errors that occur when reading from or seeking a stream
- Server rejection of an upload (a terminal state for the protocol)
- Attempting to resume a cancelled, rejected, or expired upload.

Requirements related to the Scotty protocol implementation (GAX)

GAX-R1 The GAX Libraries **must** contain an implementation of the Scotty upload protocol

The Scotty upload protocol implementation **must not** be in a separate library, and the existing libraries for Storage **must not** be used. The implementation there is for a different older version of the protocol, and adapting it is error-prone.

GAX-R2 GAX Libraries **must** expose a Scotty Helper functionality to the end-user

The GAX Libraries **should** expose Scotty Helper functionality as a class.

The Scotty Helper and Scotty Protocol functionality **must** be testable separately from the Client Libraries.

The Scotty Helper and Scotty Protocol functionality **must** follow the existing best practices for logging, observability, and actionable errors, that are implemented in the GAX library containing them.

GAX-R3 The protocol implementation **should NOT** be accessible to an end-user except via the Scotty helper

This is to keep the public surface minimal. The goal is to avoid having the details of the protocol implementation exposed as a fully public surface that we must support.

Only the API elements on the Helper that the end-user must interact with should be exposed:

- The `begin-upload` and `resume-upload` methods
- The setters and getters for the Scotty-specific configuration parameters, or the equivalents, e.g. the Builder or Factory class and methods
- The functionality allowing tracking the upload progress, e.g. the callback function and the upload progress tracking data structure
- The getters for the upload URL
- The elements necessary to reserve the namespace for cancelling and pausing the upload, such as the callback function return type.

GAX-R4 The protocol implementation **must** keep at least two chunks of data (previous and current) buffered when uploading, and to "seek" within those buffered chunks if needed (for recovery)

This is to ensure that we don't seek the stream unnecessarily, since it might be unseekable or expensive to seek. The server will typically only require seeking within either the current or the previous chunk, but this is not guaranteed.

The protocol implementation **may** keep more data buffered, e.g. buffer ahead while sending the current chunk.

GAX-R5 The protocol implementation **should** seek the stream if needed (for recovery), when the offset falls outside the buffered data

If the stream passed by the end-user is unseekable, then the protocol implementation **must** raise an error instead. Likewise if seeking the stream ends up in a language-specific error, that error **must** be raised to the end-user.

In both those cases the underlying error should be wrapped to be an actionable error.

GAX-R6 The protocol implementation **should** allow logging the (truncated) requests and responses for Scotty, along with the errors and state transitions

In fact the logging is the recommended way to test with the Scotty server fake when implementing the protocol. See Cloud SDK for Scotty: Testing the protocol implementation (link to the section).

The details here are language-specific, and are left to the language implementations.

Requirements related to Scotty generation (GEN)

GEN-R1 The Gapic Generators **must** use both Scotty annotation and a per-API Scotty setting in gapic.yaml for deciding when to generate Scotty methods

The Generators **must** read the Scotty settings in gapic.yaml to determine whether the Scotty generation is enabled for the API. If the Scotty generation is not enabled, the code for the Scotty RPCs **must not** be generated.

The Generators must use the Scotty annotation to determine whether a given RPC is a Scotty RPC. Scotty RPCs are ordinary unary methods. If a Scotty annotation is on a method that is not an unary method (streaming, LRO, paginated, etc) the Generators **must** raise an error.

GEN-R1.1 Until the annotation is published and the gapic.yaml Scotty setting is added, the generators **should** use a hardcoded allowlist

This includes every generator in scope for Milestone 1.

The allowlist will consist of a single method from Google Ads "googleads" libraries: `rpc CreateYouTubeVideoUpload` in `service YouTubeVideoUploadService`.

GEN-R2 The Gapic Generators must read the Scotty initial URL prefix from a per-API Scotty setting in gapic.yaml and use it when generating the Scotty methods

The details for using the URL prefix are in the "The Scotty initial URL and prefix" section.

GEN-R2.1 Until the gapic.yaml Scotty setting is added, the generators **should** use a hardcoded value

The value for the Google Ads "googleads" libraries is `resumable/upload`.

Addendum: Scotty documents

Teams requesting Scotty features

Cloud SDK for Scotty: the API team's and the end-user's CUJs

Cloud SDK for Scotty: Universal Resumable Upload Protocol

Cloud SDK for Scotty: Cloud SDK Libraries update requirements